Scaling Portfolio-Level Reinforcement Learning via Anchor-Token Cross-Attention

Prabal Negi
`market-agent` project
Architecture analysis and manuscript drafted with Claude (Anthropic)

September 2026

Abstract

`TradingGame` is a rule-constrained portfolio-management reinforcement learning environment: an actor-critic policy chooses buy/sell/hold actions jointly across a candidate universe of N stocks under a shared cash constraint, trained with Proximal Policy Optimization (PPO). The current policy network aggregates cross-candidate context with full self-attention over all N candidates plus a portfolio token, an $O(N^2)$ operation. This note (i) formalizes the environment and the existing architecture, (ii) reports a direct empirical characterization of that architecture's GPU memory footprint — thirteen measurements on real hardware, fitted to a closed-form model with $R^2 = 0.9999999$ — showing the design cannot reach a N=1,500-company universe at useful batch sizes, and (iii) proposes an $O(N(K+2))$ replacement, *anchor-token cross-attention*, in which each candidate attends only to a small fixed set of K real anchor stocks plus separate macro and portfolio tokens, rather than to every other candidate or to a single pooled summary. We show the redesign is necessary but not sufficient: removing the quadratic term exposes a previously-dominated linear term, whose coefficient we also measure directly, giving a closed-form bound on the batch size the redesign alone can reach at a given N.

Contents

1 Introduction **2**

2 Related Work **2**

3 Environment and Decision Problem **3**

4 Baseline Architecture: Full Self-Attention **4**

5 Empirical Characterization of the Baseline **5**
5.1 Fitted memory model 5
5.2 A caveat: peak transient memory exceeds the fitted quantity 6

6 Proposed Architecture: Anchor-Token Cross-Attention **6**
6.1 Design 6
6.2 Why not a single pooled token 6
6.3 Complexity 7
6.4 Pipeline 7

7 The Redesign Is Necessary but Not Sufficient **7**

8 Training **8**

9 Discussion and Open Questions **9**

10 Conclusion **9**

1 Introduction

`TradingGame` trains a single reinforcement-learning policy to manage a simulated portfolio of Indian (NSE/BSE) equities under an explicit rule set: a fixed starting cash balance; a 0.5% fee on every transaction; sale proceeds that settle into spendable cash only after a two-trading-day delay; a one-trading-day lock-up before a newly-bought lot can be voluntarily sold again; and an unconditional forced exit after ten trading days. The policy observes a capped *candidate universe* of N stocks at each decision step and must choose, jointly, which to hold, sell, or buy — subject to a single shared cash pool, so the decisions are coupled across candidates, not independent per-stock classifications.

The current implementation (`ActorCriticPolicy` in `packages/TradingGame/src/policy.jl`) caps N at 60. Growing the candidate universe toward index scale ($N \approx 1{,}500$) is desirable for realistic diversification, and is newly plausible now that training runs on a dedicated GPU rather than CPU alone. Section 4 shows why the current architecture cannot simply be pointed at a larger N: its cross-candidate attention mechanism is quadratic in N. Section 5 characterizes that cost directly, on real hardware, rather than by asymptotic argument alone. Section 6 derives a linear-cost replacement, and Section 7 shows why that replacement is a necessary but incomplete fix.

2 Related Work

Self-attention and its cost. Scaled dot-product self-attention [Vaswani et al., 2017] lets every element of a set condition on every other element, at $O(n^2)$ time and memory in the set size n. This is the mechanism the current `ActorCriticPolicy` uses across candidates.

Reducing attention to sub-quadratic cost. Several strategies replace all-pairs attention with attention against a smaller, fixed reference set. Lee et al. [2019] introduce the *Induced Set Attention Block* (ISAB): rather than n elements attending to all n others, they attend to $m \ll n$ learned *inducing points*, reducing cost from $O(n^2)$ to $O(nm)$ while remaining a permutation-invariant function of the input set — exactly the structure we adopt in Section 6, with the inducing points replaced by a small set of *real, interpretable* anchor stocks. Other lines of work achieve sub-quadratic attention via low-rank projection of the sequence dimension [Wang et al., 2020] or kernel approximations of the softmax [Choromanski et al., 2021]; we do not adopt these here because they trade the interpretability of "which stocks is candidate i attending to" for a compressed or randomized basis, which is a poor fit for a financial decision system where that interpretability is itself valuable.

Recurrent per-element encoders. Each candidate's own price history is encoded independently with a Gated Recurrent Unit [Cho et al., 2014] before any cross-candidate mixing; this per-candidate encoding is where the dominant *linear*-in-N cost identified in Section 7 originates, and it is unaffected by whichever cross-candidate attention pattern is used downstream.

Combined policy/value networks trained by self-play or policy-gradient RL. AlphaGo Zero and its generalization [Silver et al., 2017, 2018] share a trunk between a policy head and a value head, trained end-to-end — the same actor-critic head structure `ActorCriticPolicy` uses, though those systems operate over a fixed-size board rather than a variable, rule-constrained financial action space. We train with Proximal Policy Optimization [Schulman et al., 2017] and Generalized Advantage Estimation [Schulman et al., 2016], in the tradition of actor-critic methods such as A3C [Mnih et al., 2016]; optimization uses Adam [Kingma and Ba, 2015]. LeCun et al. [2015] give a general account of the representation learning this architecture relies on throughout.

Internal precedent. This repository's own `StockSwingPredictor` package made the analogous choice once before, for a different model: its first architecture (`DualCNN_v1`) aggregated a target stock's peer context by *mean-pooling* up to 150 peer embeddings; the second (`DualCNN_v2`) replaced that pooling with cross-attention, specifically because a flat average cannot express *which* peers matter for a given target. Section 6 applies the identical argument to `TradingGame`: an earlier draft of this proposal itself made the mean-pool mistake it is meant to avoid, by aggregating all N candidates into one pooled "market pulse" token before realizing this reintroduced exactly the selectivity loss that motivated `DualCNN_v2` in the first place.

3 Environment and Decision Problem

We formalize the rules enforced by `packages/TradingGame/src/env.jl` (`TradingGameRules.txt`, repo root) as a discrete-time Markov decision process.

Portfolio state. At trading-hour index t, the portfolio holds spendable cash C_t, a set of reserved-cash lots $\mathcal{R}_t = \{(\rho_k, \tau_k)\}$ (amount, maturity trading-day index), and a set of open lots $\mathcal{L}_t = \{(i_\ell, q_\ell, p_\ell, d_\ell)\}$ (stock index, quantity, entry price, entry trading-day index). Portfolio value is

$$V_t \;=\; C_t \;+\; \sum_{(\rho,\tau)\in\mathcal{R}_t} \rho \;+\; \sum_{(i,q,p,d)\in\mathcal{L}_t} q \cdot P_{i,t}, \tag{1}$$

where $P_{i,t}$ is stock i's price at t — rules 2 and 6 of `TradingGameRules.txt`, restated identically.

Settlement, lock-up, forced exit. Let $\delta(t)$ be the trading-day index of hour t. A reserved-cash lot (ρ, τ) matures — moves into C_t — once $\delta(t) \geq \tau$, with $\tau = \delta(t_{\text{sell}}) + \tau_{\text{settle}}$, $\tau_{\text{settle}} = 2$ (rule 4). A lot (i, q, p, d) may be voluntarily sold only once $\delta(t) - d \geq \tau_{\text{lock}}$, $\tau_{\text{lock}} = 1$ (rule 9), and is *unconditionally* liquidated once $\delta(t) - d \geq \tau_{\max}$, $\tau_{\max} = 10$ (rule 8), independent of the policy's action.

Action space. At each decision step the policy emits, jointly for every candidate $i \in \{1, \ldots, N\}$, a discrete type and a continuous weight:

$$a_t \;=\; \left(a_{i,t}^{\text{type}},\, w_{i,t}\right)_{i=1}^{N}, \qquad a_{i,t}^{\text{type}} \in \{\text{HOLD}, \text{SELL}, \text{BUY}\}, \quad w_{i,t} \in [0, 1]. \tag{2}$$

$w_{i,t}$ is only meaningful when $a_{i,t}^{\text{type}} = \text{BUY}$: it is the requested share of available cash, renormalized across every candidate selected BUY this step so that total notional-plus-fee never exceeds C_t (rule 5) — a deterministic masking layer (`resolve_actions` in `action.jl`), not something the policy must learn unaided. Every executed transaction pays a fee $\varphi = 0.005$ of its notional value (rule 10), and a SELL is only ever offered to the policy for lots that have cleared the lock-up in Eq. (2)'s constraint.

Reward. Consistent with rule 11's "continuously maximize portfolio value,"

$$r_t \;=\; \log\frac{V_t}{V_{t-1}}, \tag{3}$$

computed every hourly bar (not only on decision bars), so unrealized mark-to-market movement is credited even when the policy takes no action. Because Eq. (1) already carries reserved cash at full value throughout its settlement delay, a sale's only effect on r_t is the fee φ — the reward requires no separate transaction-cost penalty term.

4 Baseline Architecture: Full Self-Attention

`ActorCriticPolicy` (`policy.jl`, current implementation) computes an embedding $h_i \in \mathbb{R}^d$ for each candidate independently with a weight-shared GRU over its own T=120-bar, C=2-channel hourly window (normalised close and previous-day range), fuses in per-candidate news and holding-state features, and forms one additional *portfolio/macro token* m from the macro time series and the global portfolio scalars. The N+1 embeddings $\{h_1, \ldots, h_N, m\}$ are then passed through full multi-head self-attention [Vaswani et al., 2017]:

$$\mathrm{Attn}(Q,K,V) \;=\; \mathrm{softmax}\!\left(\frac{QK^\top}{\sqrt{d}}\right)V, \qquad Q = K = V = \big[h_1; \ldots; h_N; m\big] \in \mathbb{R}^{(N+1)\times d}. \tag{4}$$

Every one of the N+1 tokens is both a query and a key: the score matrix $QK^\top$ has $(N{+}1)^2$ entries, computed and differentiated once per attention head and once per batch element. This is the mechanism by which the network lets candidate i's decision condition on candidate j's current state — necessary for a *joint*, cash-constrained decision — but its cost scales as

$$\mathrm{cost}_{\mathrm{attn}}(N) \;=\; \Theta\big((N+1)^2\big). \tag{5}$$

Figure 1(a) shows this connectivity for a small illustrative N: every node connects to every other.

Game v2. The v2 rule set (`-game-version 2`) adds a *cash token*: cash is treated as a pseudo-stock and embedded from four scalars (cash and reserved-cash shares of portfolio value, utilisation of the cash cap, and time spent above it), giving N+2 tokens and $\Theta\big((N+2)^2\big)$ attention cost. The asymptotics and the conclusions below are unchanged. The GPU measurements in Section 5 were taken on the earlier N+1-token, C=3 policy; the policy now has one fewer input channel and (under v2) one extra token, which shifts the fitted coefficients slightly but not the N^2 scaling. They have not been re-run.

Game v3. The v3 rule set (`-game-version 3`, BSE at a 15-minute cadence) simplifies the network and removes every token except the stocks: the macro and news branches are switched off, the GRU is replaced by a single dense fusion layer over the stock's 70 hourly log-returns, its eight state features (held flag, unrealised P&L, hold time left, current 15-minute price, and four legality inputs: rebuy cooldown left, affordability of one share, position-cap headroom, sellable) and the seven portfolio scalars repeated beside it, and there is neither a cash nor a portfolio token. Attention therefore runs over the N stock tokens alone, $\Theta(N^2)$, and the critic reads the mean attended token together with the portfolio scalars. An auxiliary head on the attended tokens predicts which rule would refuse a purchase of that stock (cooldown, cash, free slot, position cap); it adds a per-stock $O(N)$ term

and feeds neither the action nor the value. The policy has 29,129 parameters (against 43,301 for v2). The fused per-stock vector is computed once per stock, so the dense layer costs $O(N)$; the measurements in Section 5 still describe the earlier GRU policy and have not been re-run, but the dominant N^2B attention term is unchanged in kind.

5 Empirical Characterization of the Baseline

Asymptotic complexity alone does not say where a design stops fitting a particular GPU, so Eq. (5) was measured directly rather than taken on faith. For each (N, B) pair, `ActorCriticPolicy` was constructed, moved to GPU, and run through one full forward pass, backward pass, and Adam optimizer step [Kingma and Ba, 2015] on random input of the correct shape; peak resident GPU memory was read from `CUDA.used_memory()` immediately after. All measurements are on a single NVIDIA GeForce RTX 5060 Ti (16.72 GB total, ≈15.57 GB usable after driver/OS reservation).

Table 1: Measured peak GPU memory, `ActorCriticPolicy` forward + backward + Adam step.

N	Batch B	Peak memory (GB)	Result
60	4	0.122	fits
60	8	0.250	fits
60	16	0.493	fits
60	32	0.986	fits
60	64	1.971	fits
60	128	3.941	fits
60	256	7.882	fits
500	4	1.083	fits
500	8	2.165	fits
500	16	4.329	fits
500	32	8.657	fits
500	64	–	**OOM**
1,500	4	3.631	fits
1,500	8	7.261	fits
1,500	16	–	**OOM**

5.1 Fitted memory model

The thirteen successful measurements were fit by ordinary least squares to

$$\widehat{\mathrm{mem}}(N, B) \;=\; c_0 \;+\; c_1 B \;+\; c_2(NB) \;+\; c_3(N^2 B) \qquad [\mathrm{GB}], \tag{6}$$

whose four terms correspond, respectively, to a fixed constant overhead, a per-batch-element cost independent of N (biases, small dense layers), a cost linear in both N and B (the GRU encoder and fusion/head layers, each processing $N \times B$ independent sequences), and a cost quadratic in N, linear in B (self-attention, Eq. (5), batched). The fit is

$$c_0 = 9.710 \times 10^{-4}, \qquad c_1 = 1.615 \times 10^{-5}, \tag{7}$$

$$c_2 = 5.090 \times 10^{-4}, \qquad c_3 = 6.400 \times 10^{-8}, \tag{8}$$

with $R^2 = 0.9999999$ across all thirteen points — residuals below 4×10^{-3} GB throughout, i.e. essentially exact given measurement granularity. This fit is the quantitative basis for every projection in the remainder of this note, and for the live calculator at `website/tradinggamescale.html`.

5.2 A caveat: peak transient memory exceeds the fitted quantity

Eq. (6) predicts $\widehat{\text{mem}}(1500, 16) \approx 14.5\,\text{GB}$ — comfortably under the 15.57 GB usable ceiling — yet N=1500, B=16 was *measured* to fail with an out-of-memory error. `used_memory()` reports resident memory after the step completes; it understates the *peak* memory held transiently during backpropagation through attention, when the softmax output, its gradient, and several intermediate tensors coexist. We do not attempt to model that peak directly; instead, every capacity claim in this note treats the highest value that was *empirically observed to succeed* (8.657 GB, at N=500, B=32) as the boundary of a verified-safe region, and treats predictions between that value and the raw usable ceiling as requiring direct verification before being relied on.

6 Proposed Architecture: Anchor-Token Cross-Attention

6.1 Design

Replace Eq. (4)'s all-pairs attention with attention against a small, fixed *context set* $\mathcal{C}$, built from three parts:

- **Anchor stocks.** A fixed subset $\mathcal{A} \subset \{1, \ldots, N\}$, $|\mathcal{A}| = K$ (e.g. index or sector leaders). Their embeddings $\{h_j : j \in \mathcal{A}\}$ are already computed as part of the per-candidate GRU pass over all N candidates — selecting them costs $O(1)$, with no additional encoding.
- **Macro token** m_{macro}, from the existing macro-series GRU, kept separate rather than pre-fused with portfolio state.
- **Portfolio token** m_{port}, from the global portfolio scalars, likewise kept separate. Under game v2 the cash token is a further fixed context token ($|\mathcal{C}| = K + 3$); it is not a candidate, so it adds no N-dependent cost. Game v3 has no macro, portfolio or cash token at all (the portfolio scalars enter every stock's fusion layer instead), so for v3 the context set would reduce to the K anchor stocks, $|\mathcal{C}| = K$.

$$\mathcal{C} \;=\; \{h_j : j \in \mathcal{A}\} \cup \{m_{\text{macro}}, m_{\text{port}}\}, \qquad |\mathcal{C}| = K + 2. \tag{9}$$

Every candidate, including anchors themselves, queries against this fixed set:

$$z_i \;=\; \text{Attn}\big(Q{=}h_i,\; K{=}V{=}\mathcal{C}\big), \qquad i = 1, \ldots, N. \tag{10}$$

This is precisely the Induced Set Attention Block construction of Lee et al. [2019], with the inducing points replaced by *interpretable, real* anchor stocks and two designated context tokens, rather than learned abstract vectors — a deliberate choice for a system whose outputs should remain auditable (Section 2).

6.2 Why not a single pooled token

An intermediate design — reject in favor of Eq. (9) — would replace the anchor set with a single pooled summary, $m_{\text{pool}} = \frac{1}{N}\sum_i h_i$, and let every candidate attend only to $\{m_{\text{pool}}, m_{\text{macro}}, m_{\text{port}}\}$. This is cheaper still, $O(N \cdot 3)$, but every candidate then reads the *identical* pooled vector: attention

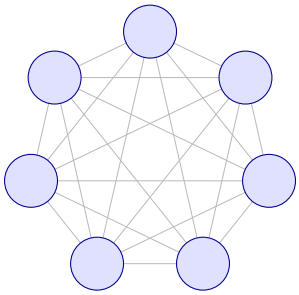
over a one-element key set degenerates to a fixed (query-independent, up to normalization) linear combination, discarding exactly the selectivity that motivates attention over mean-pooling in the first place — the same reasoning `StockSwingPredictor` applied when it replaced `DualCNN_v1`'s mean-pool with `DualCNN_v2`'s cross-attention (Section 2). Real, distinguishable anchor stocks let different candidates learn different attention weights over $\mathcal{A}$.

6.3 Complexity

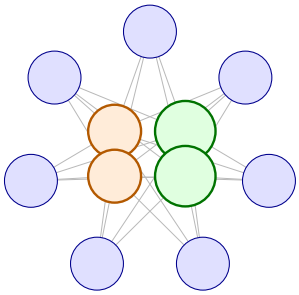
Attention cost is now

$$\text{cost}_{\text{attn}}(N, K) \;=\; \Theta\big(N \cdot (K+2)\big), \tag{11}$$

linear in N for fixed K, versus Eq. (5)'s quadratic cost. Figure 1(b) shows the resulting connectivity: outer (candidate) nodes connect only to the small central context set, not to each other.



(a) Baseline: full self-attention. Every candidate connects to every other — $\binom{N}{2}$ edges, $\Theta(N^2)$.



(b) Proposed: anchor-token cross-attention. Candidates (blue) connect only to K=2 anchors (orange) and macro/portfolio tokens (green) — $N(K+2)$ edges, $\Theta(N)$ for fixed K.

Figure 1: Attention connectivity, illustrated at a small N for legibility.

6.4 Pipeline

Figure 2 shows the full proposed forward pass. The per-candidate GRU encoder, and the actor/critic heads, are unchanged from the baseline; only the aggregation stage between them changes.

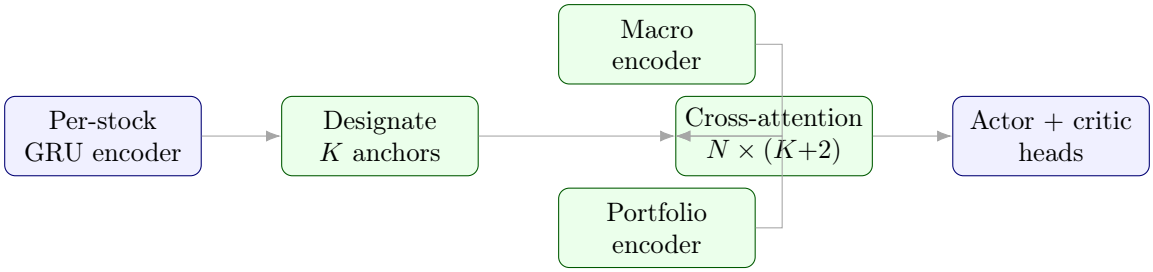


Figure 2: Proposed policy pipeline. Blue: unchanged from the baseline. Green: new or restructured relative to Section 4.

7 The Redesign Is Necessary but Not Sufficient

Substituting Eq. (11) for Eq. (5) inside the fitted model of Eq. (6) — reusing c_0, c_1, c_2 unchanged, since the GRU/fusion/head costs they capture do not depend on the attention pattern, and reusing

c_3 as the measured per-key-value-pair cost of attention itself — gives a projected memory function for the proposed design:

$$\widehat{\text{mem}}_{\text{proposed}}(N, K, B) \;=\; c_0 + c_1 B + c_2(NB) + c_3\big(N(K+2)B\big). \tag{12}$$

As a consistency check, setting $K = N - 2$ (a degenerate anchor set equal to the whole candidate universe) recovers Eq. (6) exactly, as it must: $\widehat{\text{mem}}_{\text{proposed}}(N, N{-}2, B) = \widehat{\text{mem}}(N, B)$.

Table 2 evaluates both models at N=1500, K=8. The proposed design does reduce memory — by roughly 16 % at B=32 — but *both* figures exceed this GPU's usable ceiling. The reason is visible directly in Eq. (12): at N=1500, the linear term $c_2(NB)$ already dominates the total almost completely ($c_2 \cdot 1500 \cdot 32 \approx 24.4\,\text{GB}$, versus a quadratic-term contribution, at K=8, of only $c_3 \cdot 1500 \cdot 10 \cdot 32 \approx 0.031\,\text{GB}$). Eq. (11)'s asymptotic improvement is real, but at N=1500 it has already stopped being the binding constraint.

Table 2: Baseline vs. proposed, projected at N=1500, K=8.

	Baseline (GB)	Proposed, K=8 (GB)
$B = 4$	3.63	3.06
$B = 8$	7.26	6.12
$B = 32$	29.04	24.46
$B = 256$	232.3	195.7

Solving Eq. (12) for the largest batch size under a memory budget M,

$$B_{\max}(N, K, M) \;=\; \frac{M - c_0}{c_1 + c_2 N + c_3 N(K+2)}, \tag{13}$$

gives, at N=1500, K=8: $B_{\max} \approx 11.4$ under the verified-safe budget (M=8.657 GB) and $B_{\max} \approx 20.4$ under the raw usable ceiling (M=15.57 GB, subject to the Section 5.2 caveat). Neither approaches the batch sizes (128–256) that fit easily at N=60 today. Reaching them at N=1500 requires a second lever independent of the attention redesign — most plausibly a smaller per-candidate embedding width, mixed-precision training, or gradient checkpointing through the GRU — which we leave as future work (Section 9).

8 Training

The policy is trained with Proximal Policy Optimization [Schulman et al., 2017] using Generalized Advantage Estimation [Schulman et al., 2016]; the loop is unaffected by the architecture change proposed here beyond the forward-pass cost analyzed above. Algorithm 1 summarizes the current implementation (`ppo.jl`, `train.jl`).

The continuous buy-weight $w_{i,t}$ (Eq. (2)) is used deterministically — $w_{i,t} = \sigma(\text{logit})$, renormalized by the masking layer — rather than sampled from a second distribution; PPO's log-probability ratio ρ_t is computed over the categorical a^{type} choice only. This keeps the stochastic side of a hybrid discrete/continuous action space tractable for a hand-rolled implementation, at the cost of not learning position-sizing stochastically. None of this changes under the proposed architecture, which affects only how s_t is encoded into $\ell_{i,t}$ and $V(s_t)$.

Algorithm 1 TradingGame training iteration

Roll out one full episode with the current policy, sampling $a_{i,t}^{\text{type}} \sim \text{Categorical}(\text{softmax}(\ell_{i,t}))$ per candidate; record $(s_t, a_t, \log \pi_{\text{old}}(a_t|s_t), r_t, V(s_t))_{t=1}^{T}$
Compute advantages via GAE: $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$, $\hat{A}_t = \sum_{k\geq 0}(\gamma\lambda)^k \delta_{t+k}$; normalize $\hat{A}_t$
for K_{epochs} passes over shuffled minibatches **do**
    $\rho_t(\theta) \leftarrow \exp\big(\log \pi_\theta(a_t|s_t) - \log \pi_{\text{old}}(a_t|s_t)\big)$
    $\mathcal{L}^{\text{CLIP}} \leftarrow \mathbb{E}_t\big[\min(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t)\big]$
    $\mathcal{L} \leftarrow -\mathcal{L}^{\text{CLIP}} + c_v\, \mathbb{E}_t[(V_\theta(s_t) - \hat{R}_t)^2] - c_e\, \mathbb{E}_t[\mathcal{H}(\pi_\theta(\cdot|s_t))]$
    Update θ with Adam [Kingma and Ba, 2015]
end for
Checkpoint if the (held-out, greedy) episode return improved

9 Discussion and Open Questions

- **Anchor selection.** Candidates include index constituents (e.g. Nifty 50), the top-K candidates by market capitalization, or one representative per major sector. Any choice must be a subset of the episode's own candidate universe, so no extra encoding pass is needed.
- **Anchor stability.** Fixing $\mathcal{A}$ per episode keeps K genuinely constant for Eq. (11)'s accounting; re-selecting anchors more frequently (e.g. today's top movers) could be more informative but reopens a small $O(N)$ selection cost per step and makes the attended set non-stationary within an episode.
- **The linear bottleneck (Section 7)** is, on the evidence here, the harder remaining problem: it is intrinsic to encoding N independent price histories, not to any particular cross-candidate mixing strategy.
- **GPU wiring.** `ppo.jl`/`train.jl` currently run entirely on CPU (a deliberate scoping decision for the initial PPO implementation); every measurement in this note characterizes the network in isolation, not the full rollout-and-update loop.
- **`assemble_observation` is a separate, CPU-bound cost** — a per-candidate Julia loop run once per environment step — outside the scope of the GPU analysis here and worth profiling independently as N grows.

10 Conclusion

Full self-attention across the candidate universe, as implemented today, is quadratic in N and — measured directly, not merely argued asymptotically — exhausts a 16.7 GB GPU at N=1500 between batch sizes 8 and 16. Anchor-token cross-attention, restricted to a small fixed context set of real anchor stocks plus separate macro and portfolio tokens, is linear in N and recovers exactly the selectivity a pooled-summary alternative would have discarded. It reduces memory meaningfully (Table 2) but does not, by itself, make N=1500 training cheap: a linear per-candidate encoding cost, fit from the same measurements (Eq. (6)), remains and now dominates. Both findings — the quadratic bottleneck and its linear successor — come from the same thirteen-point empirical characterization and the same closed-form model, which is the artifact we consider most durable here: not a specific proposed architecture, but a validated method for answering "does this fit on this GPU" without guessing.

References

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2021.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.

Juho Lee, Yoonho Lee, Jungtaek Kim, Adam R. Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant set functions. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97, pages 3744–3753, 2019.

Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, pages 1928–1937, 2016.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.

Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.